How a bouquet upload is saved

Student file: Acuesta_John_Constantine_Finals_LabActivity1_HOW_IT_WORKS.pdf
Theme: Petal Express (Laravel 11 + Blade + Breeze, SQLite).

This is the learning book for the whole lab.
Part 1 explains the pieces in order, including the exact lines
that display the saved WebP. Part 2 is one figure per page.

You upload a JPG, a PNG, or a WebP. Intervention writes one
watermarked WebP. Blade prints that URL in an img tag.
There is no separate thumbnail file.

Notes: docs/FINALS-LAB-ACT-1-CODE.md
Demo login: admin@test.com / 123
Show the watermark on /bouquets/{id}, not on /posts/{id}.

1. What this lab adds

Labs 2 and 3 already save a bouquet row, check the admin role, validate the form, and soft-delete the row. This lab adds the photo. The visitor may send a JPG, a PNG, or a WebP. The app does not keep that file.

Intervention Image reads the upload, shrinks it if it is wider than 1000 pixels, stamps public/images/watermark.png on the bottom-right, and encodes a new WebP. Laravel stores the path in showcase_flowers.image. Blade later prints that path inside an img tag. The browser downloads the file. Intervention does not run again on the way out.

2. Words you need

A route is the URL. POST /admin/posts is the Add bouquet form. A controller is the PHP class for that URL. Ours is AdminPostController. A form request checks the fields first. Ours is PostRequest. A model is the class for one table. Post uses the table showcase_flowers, not a table named posts.

A migration created the table. The photo column is a string named image. It holds a path, not pixels. Blade is the HTML file. A disk is Laravel's name for a folder. The public disk root is storage/app/public, set in config/filesystems.php.

3. The two files, and why the names differ

The stamp is petal-express/public/images/watermark.png. The name stays watermark.png because saveWatermarkedImage() always opens that exact path. It is part of the project. You do not upload it per bouquet. On this machine it is a PNG, 560 by 160, with a transparent background.

The bouquet photo is storage/app/public/products/ plus a UUID plus .webp. products/ keeps uploads in one folder. Str::uuid() makes a random 8-4-4-4-12 id so two files named rose.jpg cannot overwrite each other. The file is written before Post::create() knows the new row id, so the name is not the id. .webp matches Format::WEBP. The original filename is discarded.

A real row on this machine is bouquet 107, Manual PDF PNG Upload. The column stores products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp. The file command reports WebP, 800 by 800. Width stayed 800 because the upload was not wider than 1000.

4. How the browser is allowed to see storage

The web server may read the public folder. It may not read storage/app/public by itself. php artisan storage:link creates public/storage as a shortcut to storage/app/public. The browser URL is /storage/ plus the column value, with no drive letter and no storage/app prefix.

For bouquet 107 the tag src is /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp. That is why displayImageUrl() returns a path starting with /storage/ and not the full folder path.

5. What happens when you click Save bouquet

create.blade.php posts to the route admin.posts.store. enctype="multipart/form-data" is required. Without it, PHP never receives the file. The input name is image. @csrf is Laravel's check that the form came from our site.

routes/web.php runs auth and role:admin before the controller. A guest is sent to login. A member gets 403. Intervention does not run for them.

PostRequest then checks image. On create the rules are required, image, mimes jpg jpeg png webp, and max 3072. max is kilobytes, so the cap is 3 MB. On edit, required becomes nullable. A blank file input keeps the old photo. If a rule fails, Laravel returns to the form and saveWatermarkedImage() never starts.

store() does not resize the photo itself. It calls saveWatermarkedImage($request->file('image')) and puts the returned string in the image column, next to title, body, price, category_id, and auth()->id(). The redirect flash is Bouquet added. Watermark applied.

6. What Intervention does, line by line

Image::decode($file) reads the bytes. After that line the code does not care that the original was JPG or PNG. This machine's PHP has the gd extension, so Intervention uses GD. The controller does not name the driver.

scaleDown(width: 1000) shrinks the photo only when it is wider than 1000 pixels. A smaller photo keeps its size. Height keeps the same shape. This is a resize, not a crop.

public_path('images/watermark.png') opens the stamp. If the file is missing, the method throws Watermark not found before Post::create(), so you get no row and no WebP. The mark is scaled to 38 percent of the photo width and never wider than 320 pixels.

insert($watermark, 12, 12, Alignment::BOTTOM_RIGHT, 0.55) paints it. The 12s are the gap from the corner. BOTTOM_RIGHT picks the corner. 0.55 keeps 55 percent of the mark's opacity. In the library the argument is named transparency, but the GD driver treats 1 as full strength.

The path is products/ plus Str::uuid() plus .webp. encodeUsingFormat(Format::WEBP, quality: 80) makes the bytes. Storage::disk('public')->put() writes them under storage/app/public. The method returns the path. That return value is the only thing the database stores. PHP deletes the temp upload at the end of the request.

7. The lines that display the image

Saving the file does not show it. An img tag shows it. The src must be a URL. One method builds that URL for every screen that should show the upload.

```
Post.php lines 57-66
if ($this->image) {
    if (Storage::disk('public')->exists($this->image)) {
        return '/storage/'.ltrim($this->image, '/');
    }
}
return $this->galleryImageUrl();
```

Line 59 skips empty columns. Line 61 checks the file is really on the public disk. Line 62 is the display line for an upload. For bouquet 107 it returns /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp. We do not use Storage::url(), because that prefixes APP_URL. A demo at 127.0.0.1:8000 would then request localhost and the image would break.

galleryImageUrl() at line 70 runs only when there is no upload. It cycles public/images/bouquets/01.jpg through 06.jpg by row id. Those JPGs are not watermarked. Seeded rows use them.

The controller only loads the row. PageController::bouquetsShow lines 57 to 61 call findOrFail($id) and return view('pages.bouquets-show'). The method displayImageUrl() runs later, inside the Blade file, when the img tag is printed.

```
bouquets-show.blade.php line 5
<img src="{{ $bouquet->displayImageUrl() }}" alt="{{ $bouquet->title }}">
```

The double braces print the string. The browser requests that src. The shortcut public/storage serves the WebP. Intervention is not called on this request.

The public list does the same call one step earlier. bouquets-index.blade.php line 47, home.blade.php line 55, and member/favorites.blade.php line 19 pass :image="$bouquet->displayImageUrl()" into the showcase-list-item component. The tag is showcase-list-item.blade.php lines 13 to 16, src="{{ $image }}", with classes h-20 w-20. Those classes are the whole thumbnail. They shrink the same WebP in the browser.

The admin list is admin/posts/index.blade.php line 27, :image="$post->displayImageUrl()". Trash repeats it at trash.blade.php line 21. admin-post-list-item.blade.php line 9 passes that URL into admin-photo-thumb. The tag is admin-photo-thumb.blade.php line 16, src="{{ $src }}". Edit uses that component at edit.blade.php line 44 with size lg. The button click on line 14 sends the same src into the modal. The modal img is admin-photo-preview-shell.blade.php lines 34 to 37, :src="previewSrc". Still one file.

Do not demo the member page /posts/{id} for the watermark. posts/show.blade.php line 9 calls $post->galleryImageUrl(), not displayImageUrl(). That page shows a packed JPG even when the row has a WebP. The public URL /bouquets/{id} is the page that uses line 5 above.

8. Replace, trash, and delete the file

Edit with no new file leaves the column alone. hasFile('image') is false, so Intervention does not run. Edit with a new file writes the new WebP first, updates the row, then deletes the old path. If the update throws, the new file is deleted and the old path stays. The row never points at a file that was already erased.

Delete on the list calls destroy(), which sets deleted_at. The WebP stays, so Trash can still call displayImageUrl(). Delete forever copies the path, removes the row, then Storage::disk('public')->delete() on that products/<uuid>.webp path. The route uses withTrashed() so Laravel can still find a trashed row. Restore does not move a file.

9. What the test proves

php artisan test --filter=LabActFourInterventionTest uses a fake public disk. It posts a JPG and asserts the stored path starts with products/ and ends with .webp. A second test soft-deletes and asserts the file remains. A third force-deletes and asserts the file is gone. The test does not look at watermark pixels. You show the mark with your eyes on /bouquets/{id} or in the admin preview.

10. What to say if you forget a line

The stamp is public/images/watermark.png. The saved photo is storage/app/public/products/<uuid>.webp. The column stores products/<uuid>.webp. The display line is Post.php line 62. The public img tag is bouquets-show.blade.php line 5. The small photo is that same URL with a small CSS box. /posts/{id} is the wrong page for this lab.

There are two picture files. public/images/watermark.png is the shared stamp. Its name stays watermark.png because AdminPostController always opens that path. storage/app/public/products/<uuid>.webp is the bouquet photo Intervention just encoded. A new UUID is used so a second upload cannot overwrite the first. The original JPG or PNG name is thrown away.

```
Two picture files

Stamp (one shared file, committed with the app)
  petal-express/public/images/watermark.png
  PNG, 560 x 160, transparent background

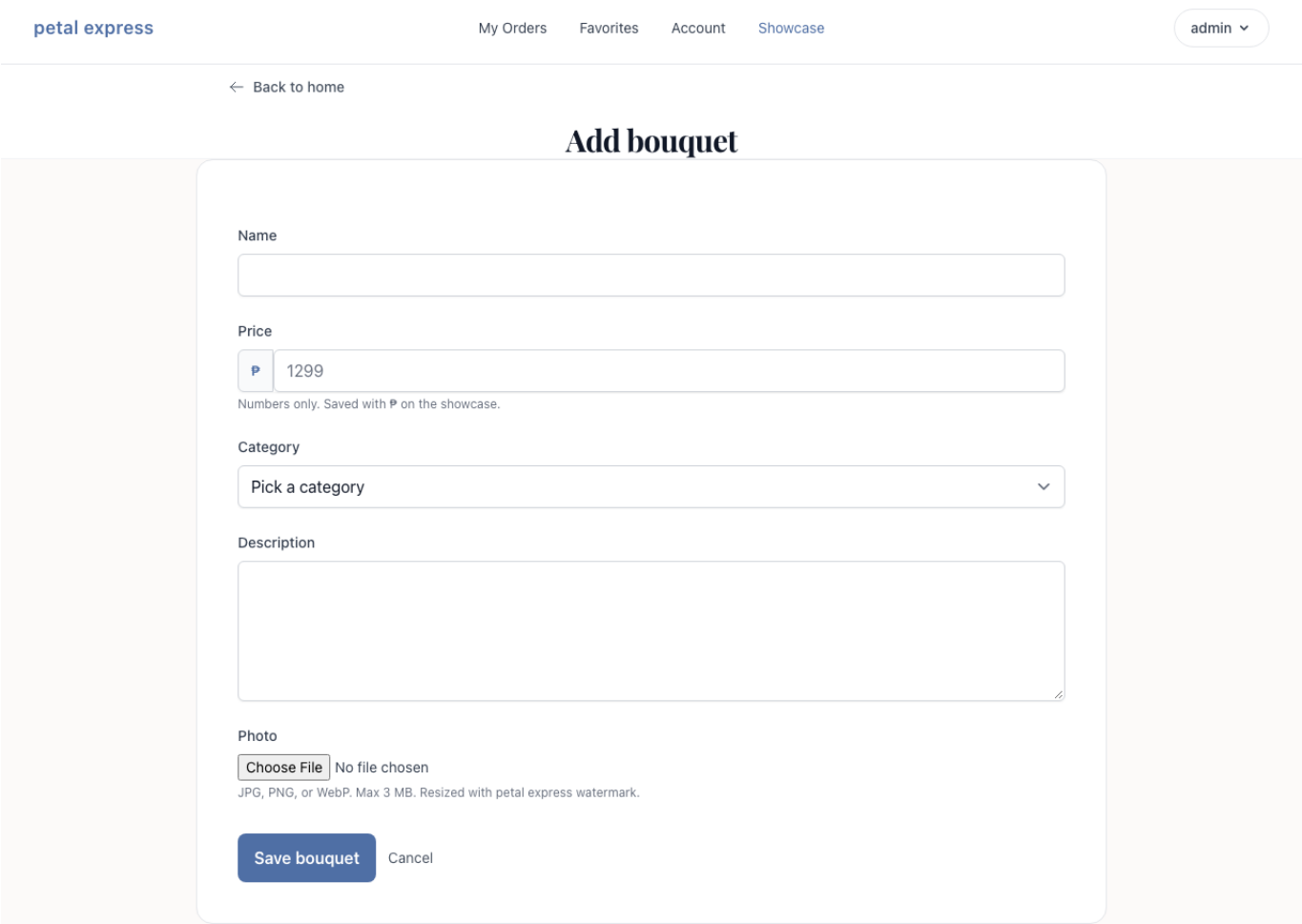
Saved bouquet photo (new file on every upload)
  petal-express/storage/app/public/products/<uuid>.webp
  example: products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp
  file(1) says: WebP image, 800 x 800

These are not the same file.
The stamp is not copied into products/.
The upload's original name is not kept.
```

This is the actual stamp file, public/images/watermark.png. It is a PNG with a transparent background, 560 by 160 pixels on this machine. It is part of the project, not an upload. saveWatermarkedImage() scales it to at most 38 percent of the photo width, and never wider than 320 pixels, then paints it 12 pixels in from the bottom-right at 55 percent opacity.



The Add bouquet form posts to admin.posts.store. The Blade file is resources/views/admin/posts/create.blade.php. enctype="multipart/form-data" is required or PHP never receives the file. The input name is image, which is the key PostRequest validates. @csrf is Laravel's check that the form came from our own site.



PostRequest runs before the controller saves anything. On create, image is required. Allowed types are JPG, PNG, and WebP. max:3072 means 3 MB. On edit the rule is nullable, so a blank file input keeps the old photo. A failed rule redirects back to the form. saveWatermarkedImage() does not run, so no WebP is written.

Step 4 — PostRequest image validation

```
// PostRequest.php
  21 |     public function rules(): array
  22 |     {
  23 |         $imageRules = $this->isMethod('put') || $this->isMethod('patch')
  24 |             ? ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072']
  25 |             : ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:3072'];
  26 |
  27 |         return [
  28 |             'title' => ['required', 'string', 'max:255'],
  29 |             'body' => ['required', 'string'],
  30 |             'price' => ['required', 'integer', 'min:0'],
  31 |             'category_id' => ['required', 'integer', 'exists:categories,id'],
  32 |             'image' => $imageRules,
  33 |         ];
  34 |     }
```

saveWatermarkedImage() is private on AdminPostController. Image::decode() reads the upload so JPG and PNG become the same in-memory picture. scaleDown(width: 1000) shrinks only photos wider than 1000 pixels. insert() stamps the watermark. encodeUsingFormat(Format::WEBP, quality: 80) makes the bytes. Storage::disk('public')->put() writes them. The method returns the relative path. store() saves that string in showcase_flowers.image.

Step 5 — saveWatermarkedImage() core

```
// AdminPostController.php
 148 |     // resize, watermark, save as webp under storage/app/public/products
 149 |     private function saveWatermarkedImage(UploadedFile $file): string
 150 |     {
 151 |         $image = Image::decode($file);
 152 |         $image->scaleDown(width: 1000);
 153 |
 154 |         $watermarkPath = public_path('images/watermark.png');
 155 |         if (! is_file($watermarkPath)) {
 156 |             throw new \RuntimeException('Watermark not found: public/images/watermark.png');
 157 |         }
 158 |
 159 |         $watermark = Image::decode($watermarkPath);
 160 |         // bigger watermark: up to 38% of photo width, capped at 320px
 161 |         $watermarkWidth = min(
 162 |             320,
 163 |             max(1, (int) ($image->width() * 0.38))
 164 |         );
 165 |         $watermark->scaleDown(width: $watermarkWidth);
 166 |
 167 |         $image->insert(
 168 |             $watermark,
 169 |             12,
 170 |             12,
 171 |             Alignment::BOTTOM_RIGHT,
 172 |             0.55
 173 |         );
 174 |
 175 |         $path = 'products/'.Str::uuid().'.webp';
 176 |         $encoded = $image->encodeUsingFormat(
 177 |             Format::WEBP,
 178 |             quality: 80
 179 |         );
```

The column does not store pixels. It stores products/ plus a UUID plus .webp. products/ keeps bouquet photos in one folder on the public disk. Str::uuid() runs before Post::create(), so the name cannot be the new row id yet. .webp matches Format::WEBP. The visitor filename is not used, because rose.jpg can be uploaded twice and can contain spaces.

Why the saved name looks like that

```
Stored in showcase_flowers.image:
  products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

products/     folder on the public disk
0f95b48d-2d8d-4a6e-8406-0aa6522e34ec
              Str::uuid()  (8-4-4-4-12 hex)
.webp         Format::WEBP, quality 80

Not used: the visitor filename (rose.jpg)
Not used: the database id
Why: two uploads must not overwrite each other,
and the file is written before the new id exists.

Browser URL:
  /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp
```

The public disk root is storage/app/public, from config/filesystems.php. Browsers cannot read that folder until php artisan storage:link creates public/storage as a shortcut. The URL is /storage/ plus the column value. Row 107 stores products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp for Manual PDF PNG Upload. displayImageUrl() returns /storage/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp.

Symlink and the database path

```
$ ls -ld public/storage
public/storage -> storage/app/public

Disk root:  storage/app/public
Web prefix: /storage
Shortcut:   public/storage   (php artisan storage:link)

sqlite> SELECT id, title, image FROM showcase_flowers
        WHERE id = 107;
107 | Manual PDF PNG Upload
    | products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

The column stores the path. It does not store pixels.
```

This figure is that saved file itself, not a screenshot of the page. The file command reports WebP, 800 by 800. Width stayed 800 because scaleDown only shrinks photos wider than 1000. The mark is in the bottom-right. This is the only image file Intervention wrote for that upload.



There is no thumbnail file. The admin list, the edit page, the preview modal, and the public detail page all use Post::displayImageUrl(). CSS makes the admin box small. The modal and the detail page show the same URL larger, which is why you can see the watermark there. One file means force delete has one path to remove.

```
The thumbnail is the same WebP drawn small

 Intervention writes ONE file:
   storage/app/public/products/0f95b48d-2d8d-4a6e-8406-0aa6522e34ec.webp

 There is no products/<uuid>-thumb.webp.

 Admin list box:  about 80px, same URL
 Edit thumb:      same URL
 Preview modal:   same URL, drawn larger
 Public detail:   same URL, min-height 420px

 Method they all call:
   Post::displayImageUrl()
   return '/storage/' + image column
```

These are the lines that display the photo. Post.php line 62 returns /storage/ plus the column. bouquets-show.blade.php line 5 puts that return value in the img src on the public detail page. Lists pass the same call into a component, and the component's img tag is what the browser paints. posts/show.blade.php line 9 is the exception. It calls galleryImageUrl(), so /posts/{id} shows a packed JPG, not the watermarked WebP.

Lines that put the WebP on the page

```
Post.php line 62, the URL for an upload:
  return '/storage/'.ltrim($this->image, '/');

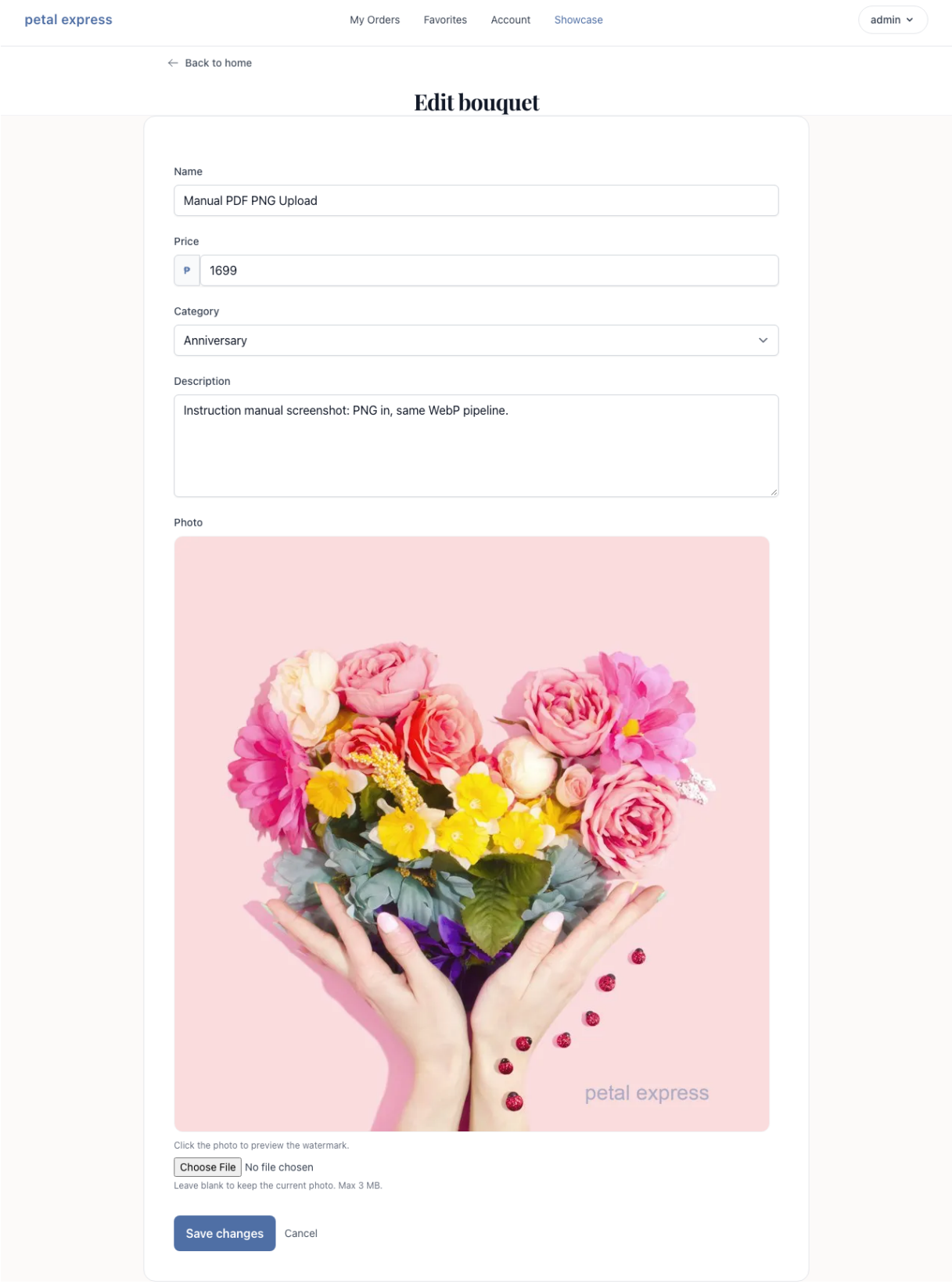
Public detail, bouquets-show.blade.php line 5:
  <img src="{{ $bouquet->displayImageUrl() }}" ...>

Public list, bouquets-index.blade.php line 47:
  :image="$bouquet->displayImageUrl()"
Then showcase-list-item.blade.php line 14:
  <img src="{{ $image }}" class="h-20 w-20 ...">

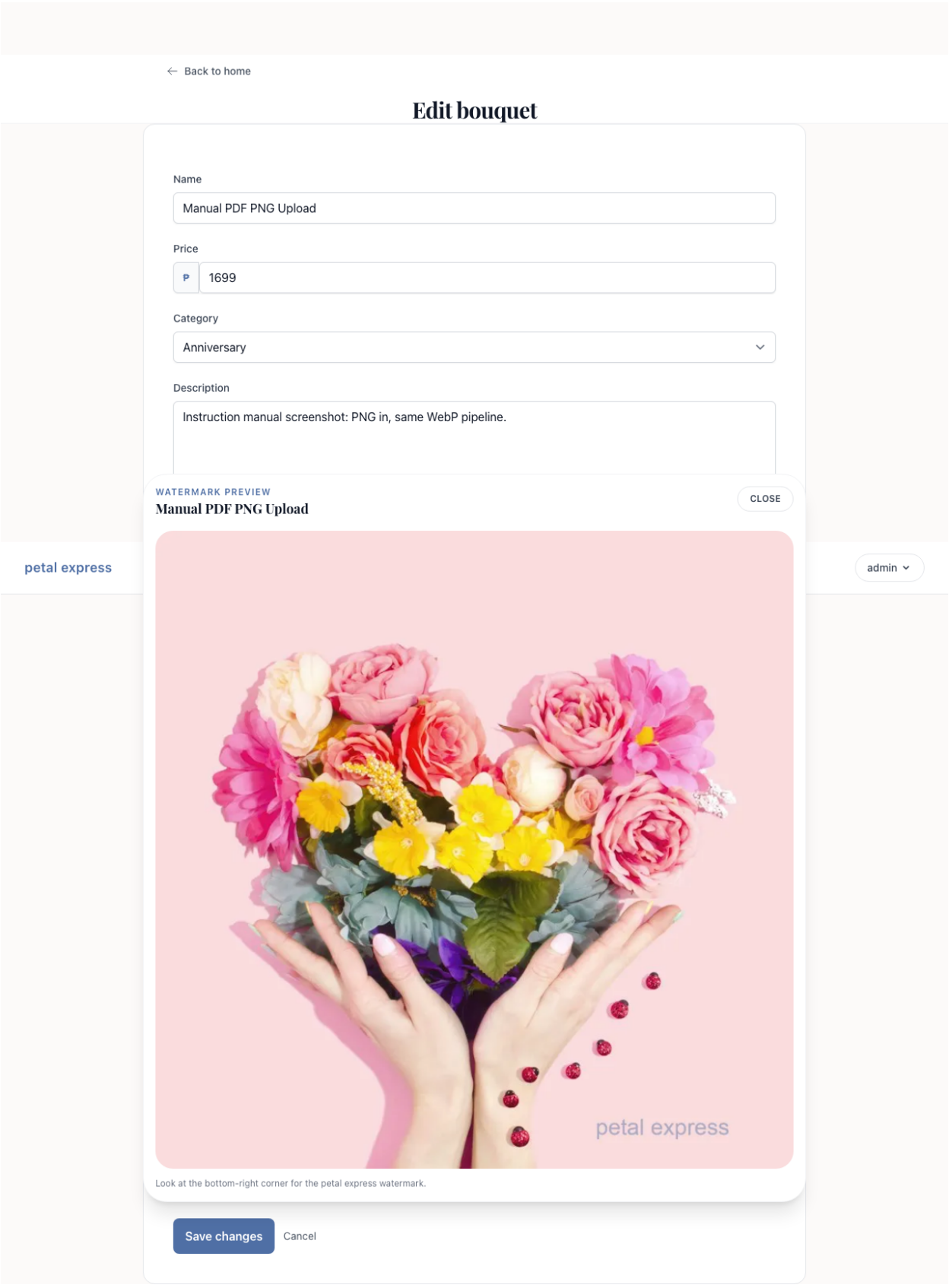
Admin list, admin/posts/index.blade.php line 27:
  :image="$post->displayImageUrl()"
Then admin-photo-thumb.blade.php line 16:
  <img src="{{ $src }}" ...>

Do not demo /posts/{id}.
posts/show.blade.php line 9 uses galleryImageUrl().
```

The edit screen looks like a thumbnail. The img src is still /storage/products/<uuid>.webp. admin-photo-thumb only changes the box size. Click it and Alpine opens the preview with that same src. Seeded bouquets with an empty image column do not use this file. They fall back to public/images/bouquets/01.jpg through 06.jpg, which are not watermarked.



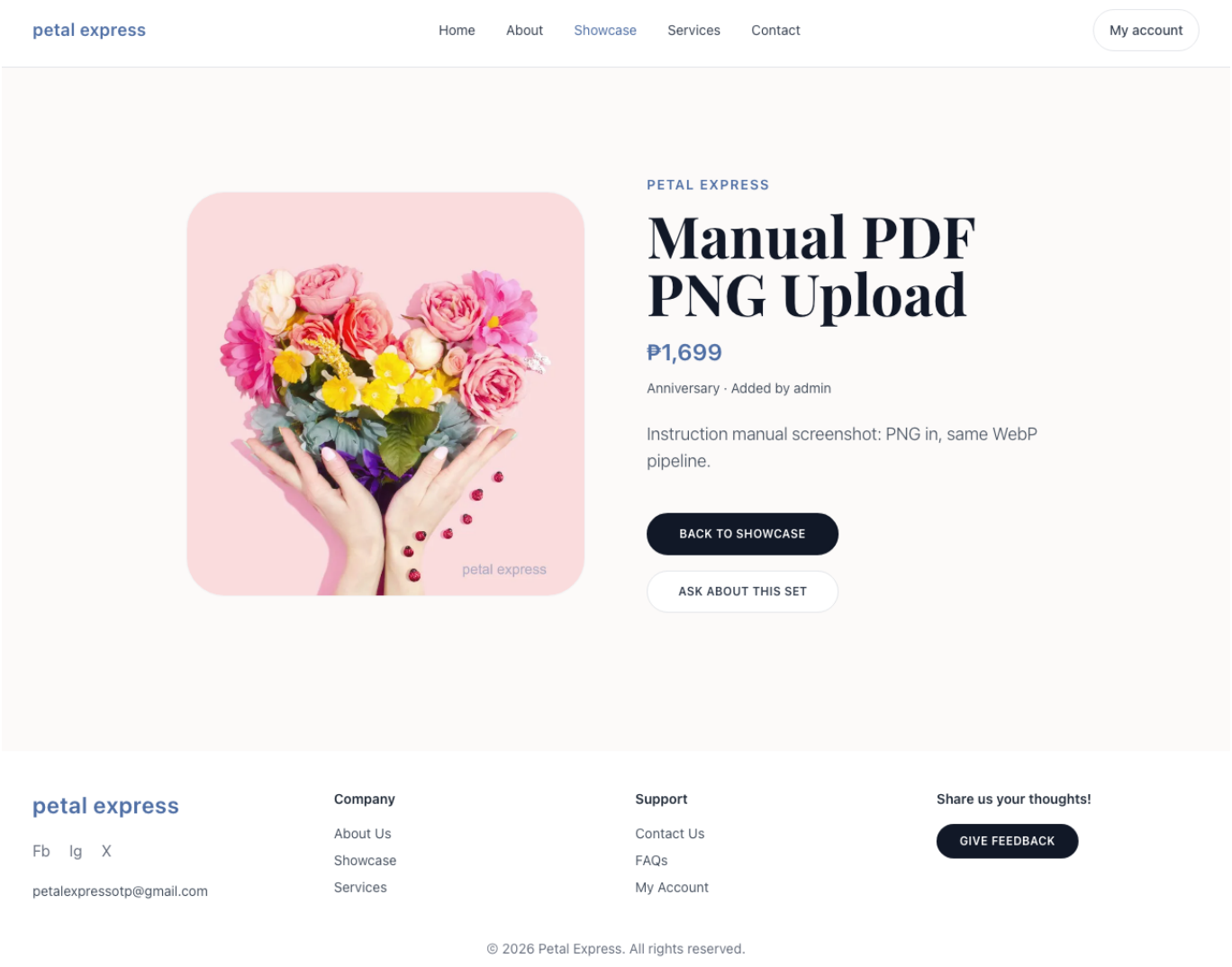
The preview modal loads the same WebP. It does not ask Intervention to render again. Intervention already burned the mark into the file at save time. The helper text tells you to look at the bottom-right corner.



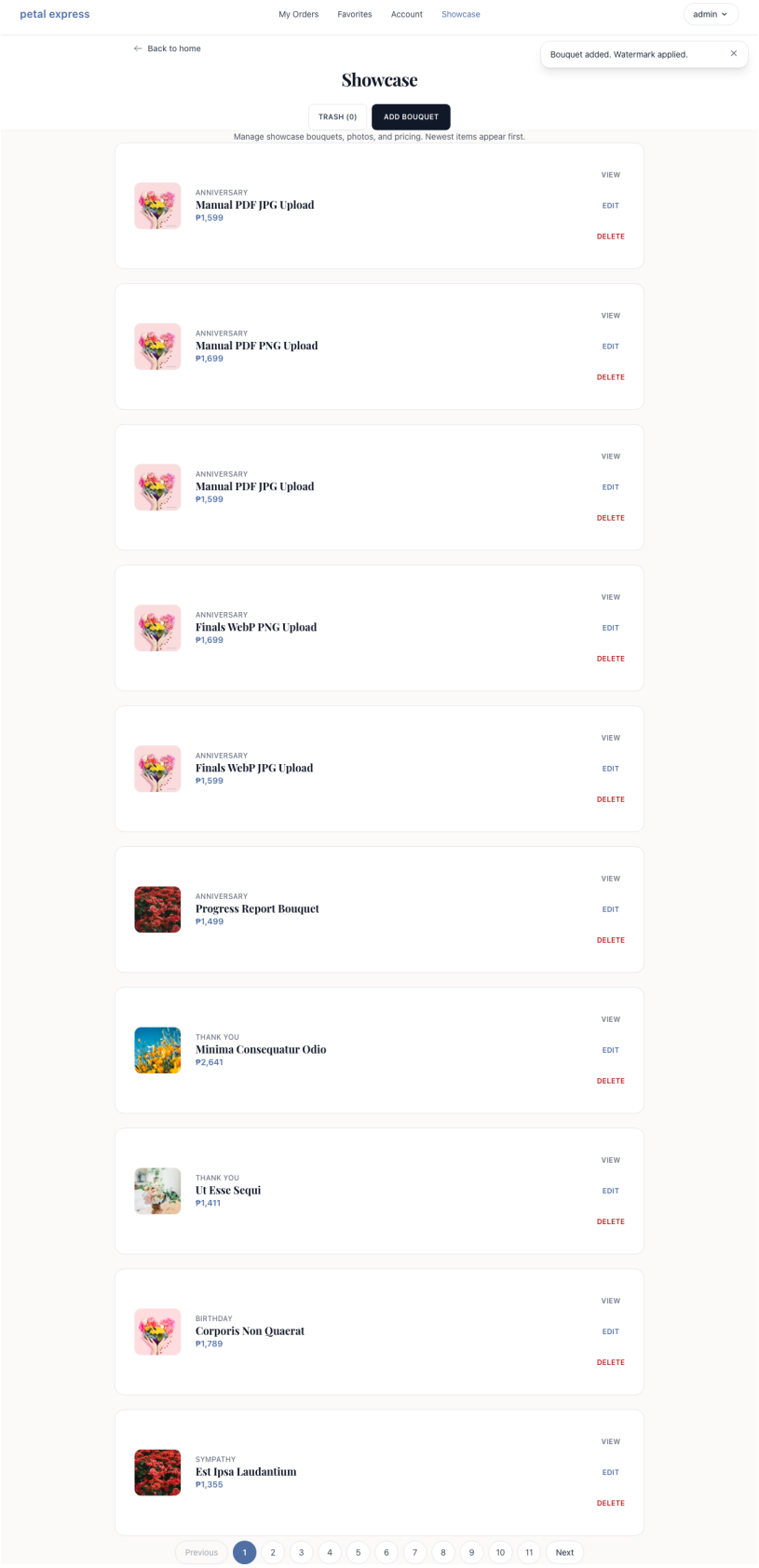
Close-up of the saved WebP. The mark is the contents of watermark.png after scaleDown and insert(). Offset 12, 12 is the gap from the bottom-right corner. 0.55 keeps 55 percent of the mark's opacity. The GD driver in Intervention treats 1 as full strength.



The public detail page calls displayImageUrl() and puts it in an img tag. The page does not decode, resize, or watermark. If the photo looks unmarked, you opened a seeded row whose image column is empty. Open a bouquet you uploaded, such as Manual PDF PNG Upload.



After a successful create, store() redirects to the admin list. The flash text is Bouquet added. Watermark applied. That text is the signal that saveWatermarkedImage() returned a path and Post::create() wrote the row. The original JPG is not in storage/app/public/products. Only the WebP is.



Delete on the list calls destroy(), which sets deleted_at. The WebP stays in storage/app/public/products, so Trash can still show it. Delete forever calls forceDelete(), which removes the row and then Storage::disk('public')->delete() on that products/<uuid>.webp path. Restore does not move a file, because the file never left.

